仮想のマンション内部シェアWEBサービス

ナヌミー

要件定義書

2026年(令和8年)4月

チェウソク

目次

1. 業務フロー図 …2p

2. ユースケース図 …3p

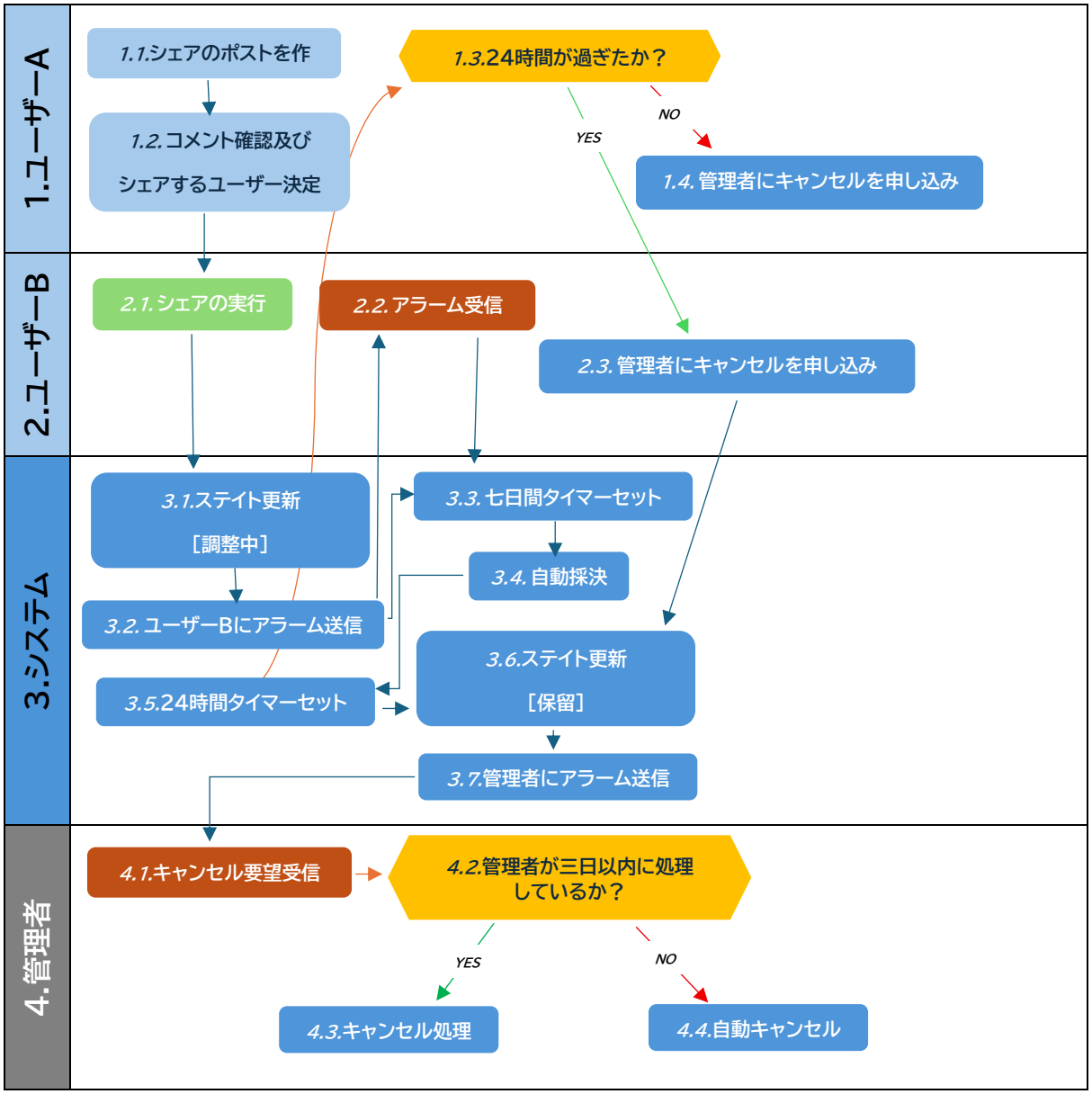
3. システム構成図 …4p

4. 画面遷移図 …5p

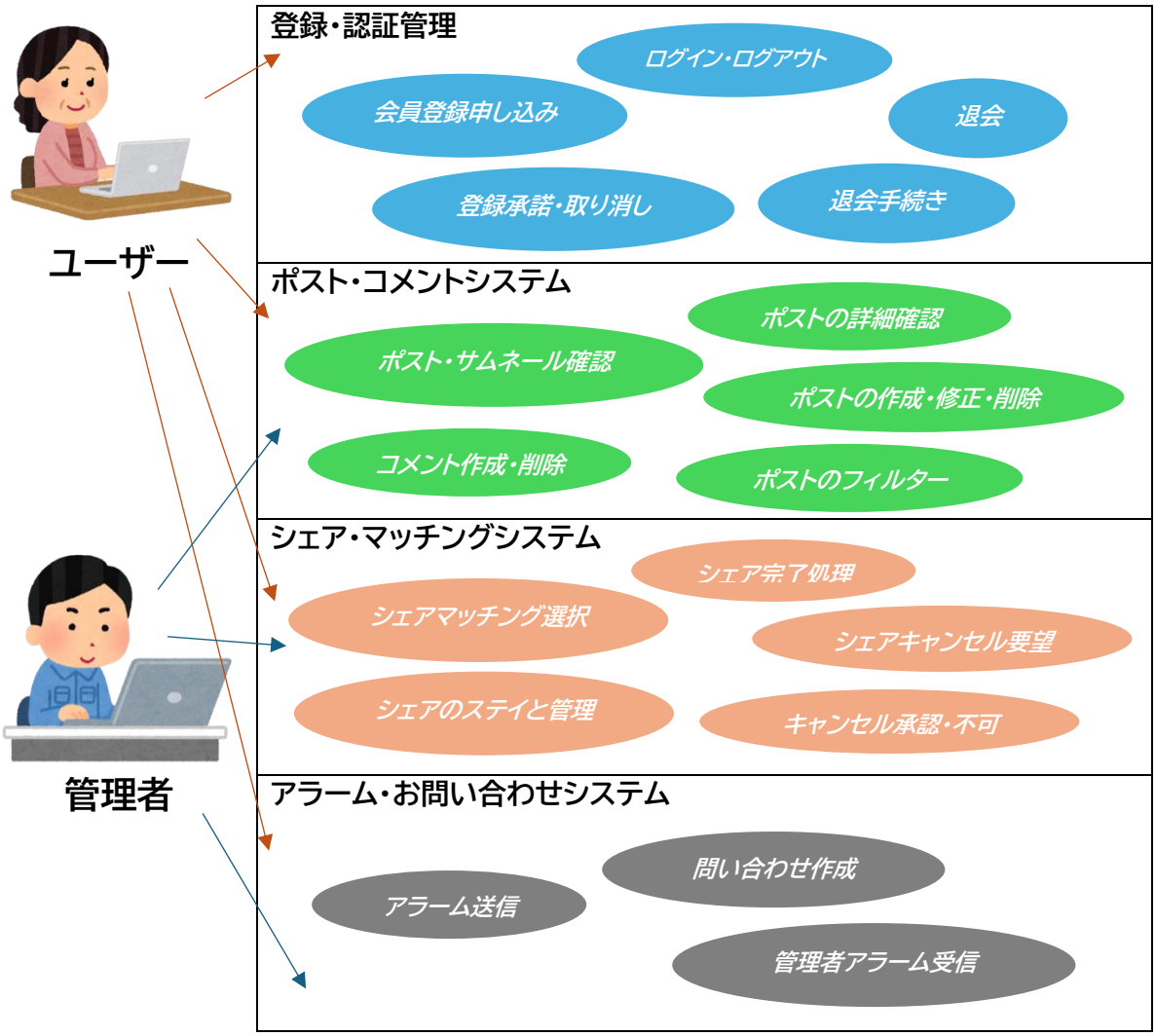
5. 概念データモデル …6p

6. 非機能要件定義書 …7p

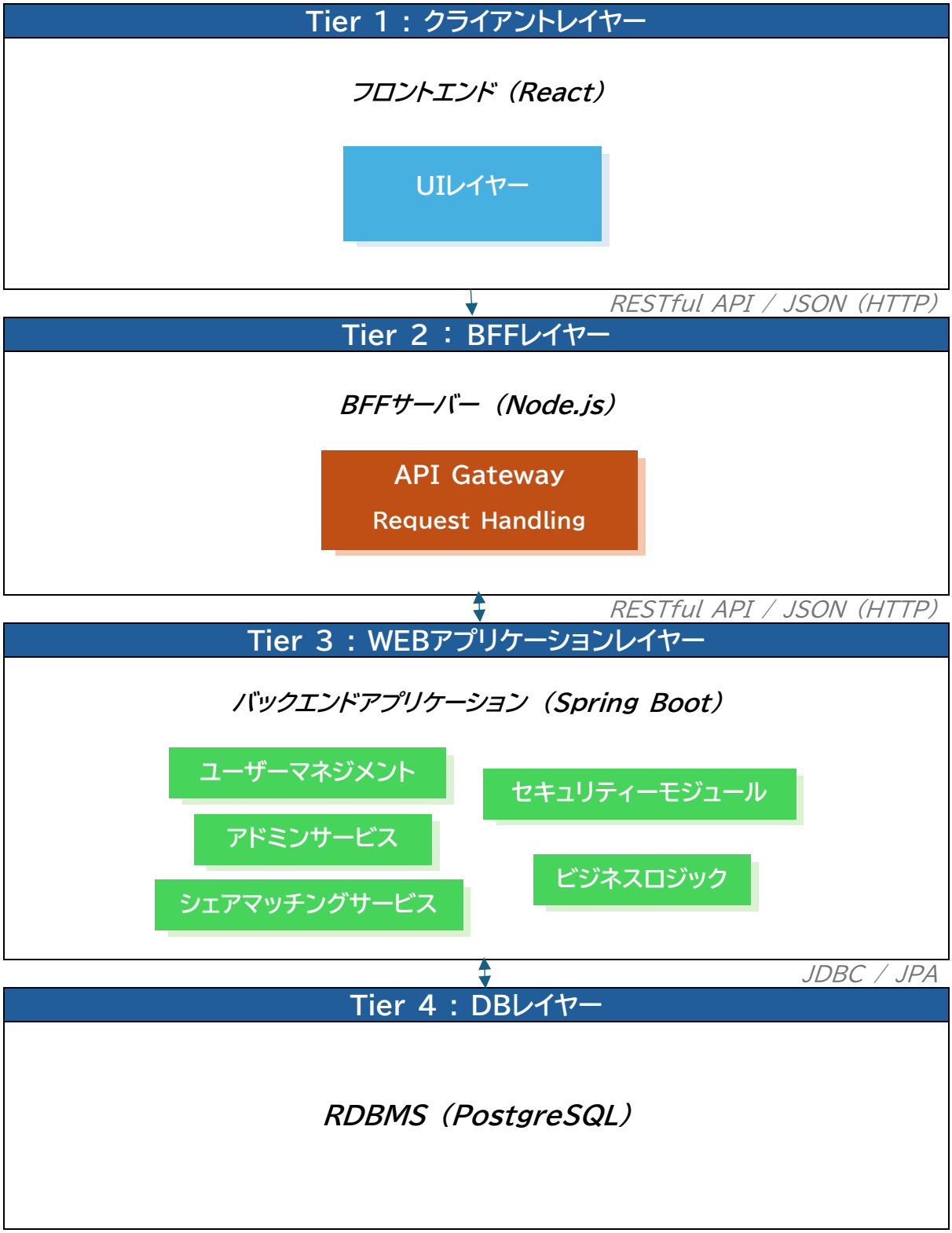
1. 業務フロー図


1.ユーザーA
2.ユーザーB
3.システム
4.管理者
1.1.シェアのポストを作
1.2.コメント確認及び
シェアするユーザー決定
1.3.24時間が過ぎたか？
YES
NO
1.4.管理者にキャンセルを申し込み
2.1.シェアの実行
2.2.アラーム受信
2.3.管理者にキャンセルを申し込み
3.1.ステイト更新
［調整中］
3.2.ユーザーBにアラーム送信
3.3.七日間タイマーセット
3.4.自動採決
3.5.24時間タイマーセット
3.6.ステイト更新
［保留］
3.7.管理者にアラーム送信
4.1.キャンセル要望受信
4.2.管理者が三日以内に処理
しているか？
YES
NO
4.3.キャンセル処理
4.4.自動キャンセル

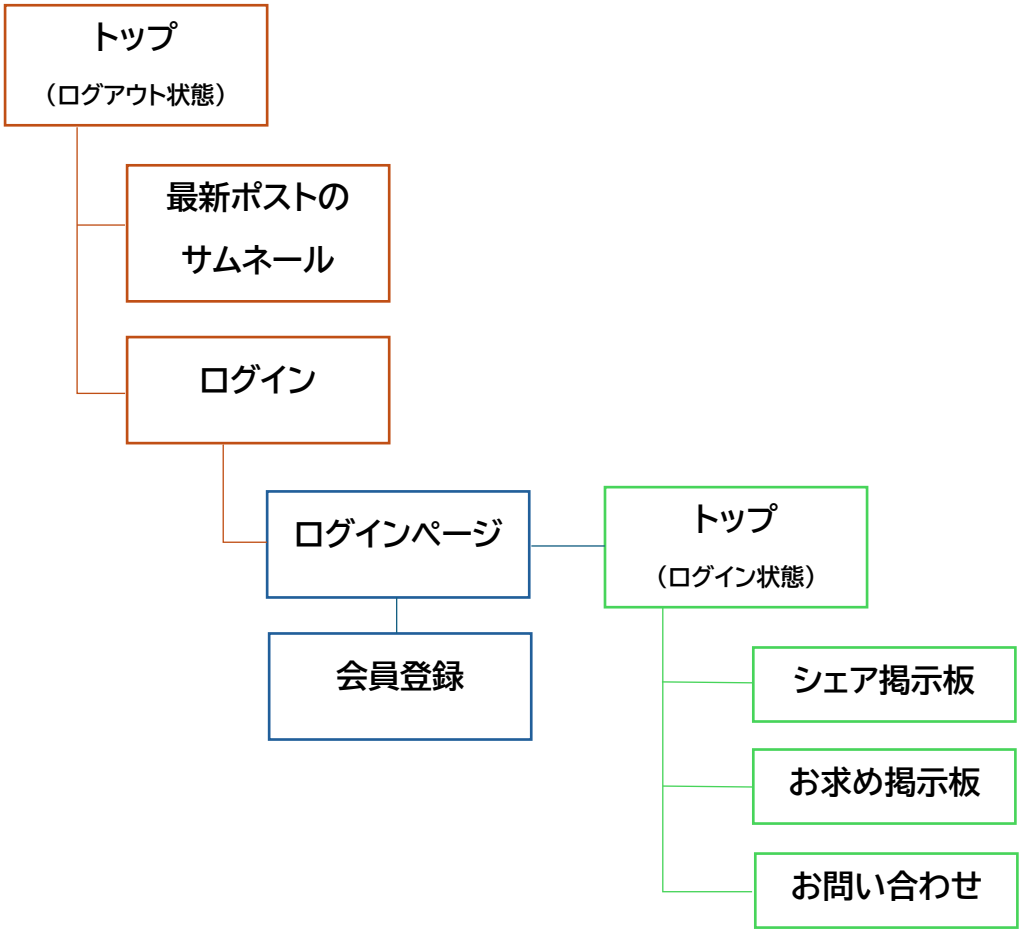
2. ユースケース図


ユーザー
管理者
登録・認証管理
ログイン・ログアウト
会員登録申し込み
退会
登録承諾・取り消し
退会手続き
ポスト・コメントシステム
ポストの詳細確認
ポスト・サムネール確認
ポストの作成・修正・削除
コメント作成・削除
ポストのフィルター
シェア・マッチングシステム
シェア完了処理
シェアマッチング選択
シェアキャンセル要望
シェアのステイと管理
キャンセル承認・不可
アラーム・お問い合わせシステム
問い合わせ作成
アラーム送信
管理者アラーム受信

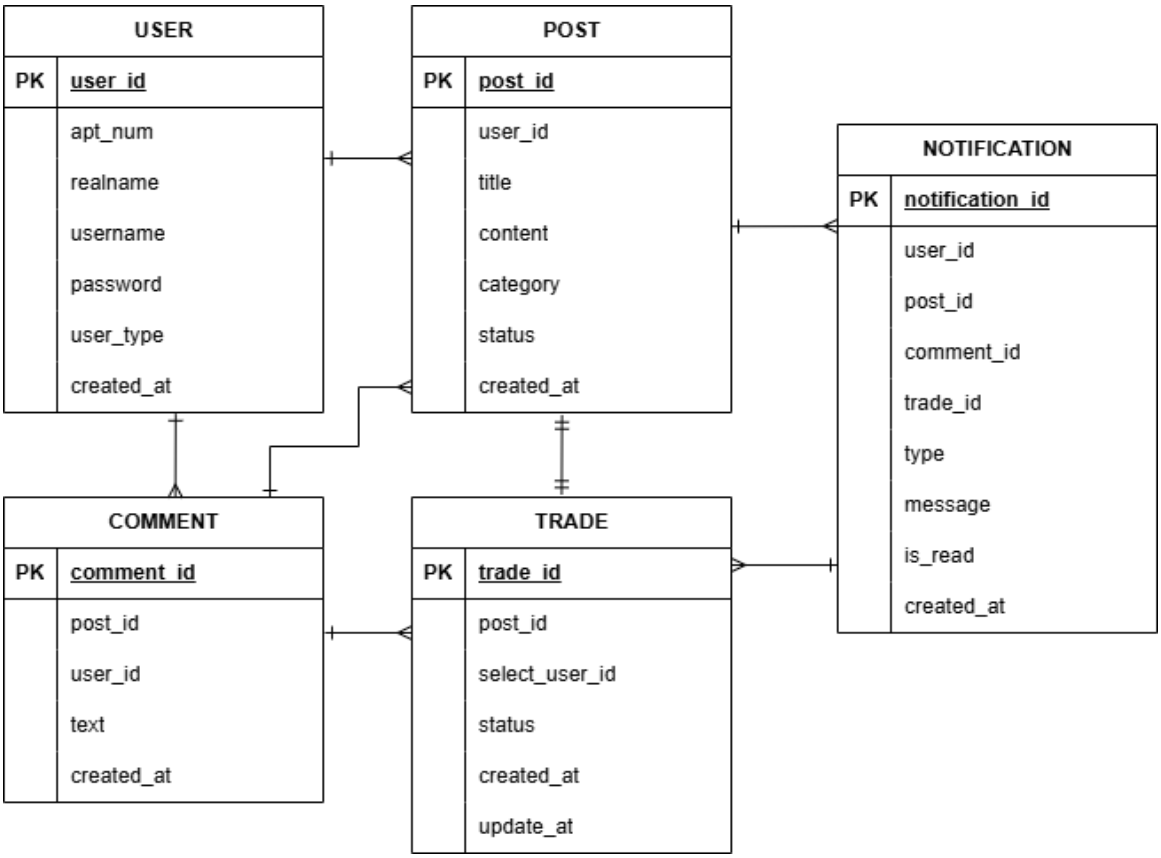
3. システム構成図


Tier 1：クライアントレイヤー
フロントエンド（React）
UIレイヤー
RESTful API / JSON（HTTP）
Tier 2：BFFレイヤー
BFFサーバー（Node.js）
API Gateway
Request Handling
RESTful API / JSON（HTTP）
Tier 3：WEBアプリケーションレイヤー
バックエンドアプリケーション（Spring Boot）
ユーザーマネジメント
セキュリティーモジュール
アドミンサービス
ビジネスロジック
シェアマッチングサービス
JDBC / JPA
Tier 4：DBレイヤー
RDBMS（PostgreSQL）

4. 画面遷移図


トップ
（ログアウト状態）
最新ポストの
サムネール
ログイン
ログインページ
トップ
（ログイン状態）
会員登録
シェア掲示板
お求め掲示板
お問い合わせ

5. 概念データモデル


USER
PK
user_id
apt_num
realname
username
password
user_type
created_at
POST
PK
post_id
user_id
title
content
category
status
created_at
NOTIFICATION
PK
notification_id
user_id
post_id
comment_id
trade_id
type
message
is_read
created_at
COMMENT
PK
comment_id
post_id
user_id
text
created_at
TRADE
PK
trade_id
post_id
select_user_id
status
created_at
update_at

6. 非機能要件定義書

大分類	中分類	詳細要件および設計仕様
1. 可用性	1. 1. システム稼働	ステータス自動遷移: 取引締結後、7日間操作がない場合、バッチ処理により自動的に「取引完了」状態へ更新し、データの整合性を維持する。
	1. 2. 障害許容力	承認プロセスの自動化: 管理者の不在時(申請から3日経過時)にシステムが自動で承認を行い、サービス滞留を防止する。
2. 性能・拡張性	2. 1. レスポンスタイム	BFF (Backend For Frontend) パターンの採用: NginxおよびNode.jsを利用し、フロントエンドに最適化されたデータを提供することで応答速度を最大化する。
	2. 2. スループット	ページング処理: 一覧表示時の取得件数を10件単位に制限し、サーバーリソースの消費抑制およびトラフィック制御を行う。
	2. 3. リソース制限	画像アップロード制御: 1投稿につき最大5枚、1枚あたり5MB以下、形式はPNG/JPGに制限し、ストレージおよびネットワーク帯域を効率的に管理する。
3. システム構成	3. 1. 階層構造	4-Tier Architecture: クライアント(React)、BFF(Node.js)、Web App(Spring Boot)、DB(PostgreSQL)の4層分離により、保守性とセキュリティを確保する。
	3.2. 通信プロトコル	RESTful API: 標準的なHTTP方式を用いたステートレスなクライアント・サーバー間通信を実装する。
	3.3. リバースプロキシ	Nginx構成: BFFパターンの実現のため、Nginxをリバースプロキシとして配置し、負荷分散およびバックエンドの隠蔽化を行う。
4. セキュリティ	4.1. 認証・認可	JWTベース認証: Spring SecurityとJWTを活用してユーザーセッションを管理し、不正なAPIアクセスを遮断する。
	4.2. アクセス制御	権限別機能制限: 未ログインユーザーに対する詳細情報・コメントの閲覧制限(マスキング)および管理者専用ページのアクセス権限を厳格に分離する。
	4.3. インフラセキュリティ	管理者情報はDBに事前登録。会員登録時には管理費入金明細等の証跡確認フローを設け、未認証ユーザーの侵入を防止する。
5. データ管理	5.1. データ永続性	Spring Data JPA: Java標準のORMを使用して、DBスキーマとオブジェクト間の一貫性を維持し、トランザクションを適切に管理する
	5.2. コンテンツ管理	**サムネール自動生成:** アップロードされた画像の1枚目をシステム側で自動的にサムネ―ルとして指定し、一覧表示を最適化する。
6. 運用・保守	6.1. 管理ツール	管理者専用コンソール: ブラウザベースの管理機能を提供し、会員の承認・退会、投稿・コメントの削除、取引状態の強制変更等の運用操作を可能にする。
	6.2. 通知システム	リアルタイム通知: コメント登録、取引締結・取消、管理者へのリクエスト発生時にベル型UIを通じて即時フィードバックを提供し、アクティブ率を向上させる。